

PULSEGUARD TECHNOLOGIES INC. - STANDARD OPERATING PROCEDURE

Document ID: SOP-ENG-001
Title: MANDATORY ENGINEERING CODE REVIEW AND QUALITY CONTROL PROCESS
Version: v3.4
Effective Date: January 15, 2026
Review Date: January 15, 2027
Department: Engineering
Author: Marcus Vance (Principal Systems Architect)
Approver: Sneha Deshmukh (VP of Engineering)
Classification: Internal Restricted

1. PURPOSE & OBJECTIVE

This Standard Operating Procedure (SOP) defines the mandatory processes, checklists, and approval workflows for code review (Pull Request) activities within the Engineering department at PulseGuard Technologies. The objective is to maintain extreme system stability, guarantee security compliance, ensure structural consistency, and verify full alignment with our enterprise standards before any code is merged into trunk branches or deployed.

2. SCOPE

This procedure is strictly applicable to all software development activities, including but not limited to frontend applications, backend services, database migration scripts, configuration-as-code files, and automated deployment definitions. It applies to all full-time engineers, contractors, third-party vendors, and temporary staff operating under the PulseGuard tech domain.

3. ROLES AND RESPONSIBILITIES

- 3.1. Code Author (Submitter)
 - Pre-tests changes locally to verify functionality.
 - Fixes all compiler warnings and static analysis errors before submission.
 - Provides complete, detailed context, screenshots/recordings, and description.
 - Resolves feedback constructively and updates documentation.
- 3.2. Peer Reviewer
 - Performs line-by-line inspection of proposed modifications.
 - Validates compliance with code conventions and safety guidelines.
 - Verifies automated build/test outcomes.
 - Refuses approval for submissions failing to meet review criteria.
- 3.3. Engineering Manager / Lead
 - Resolves technical gridlocks and disputes between author and reviewer.
 - Audits review metrics (such as review duration and depth).
 - Handles compliance escalations regarding bypassed quality processes.

4. DEFINITIONS AND REFERENCES

- Pull Request (PR): A formal submission requesting integration of code changes.
- Trunk Branch: Main, production, or develop branches used as release baselines.
- SAST: Static Application Security Testing (automated code vulnerability scanner).
- Core Security Scope: Any changes affecting authentication, authorization, encryption, payment gateways, data validation, or external interfaces.
- SOP-ENG-002: Security Incident Response Protocol (reference document).

5. PRE-SUBMISSION PREREQUISITES

Before a developer may open a Pull Request, the following steps must be completed:

- 5.1. The author must execute local builds without compiler warnings.

- 5.2. Unit tests must run locally and pass at 100%.
- 5.3. A local static analysis scan (Lint/Prettier) must be executed.
- 5.4. A secret-scanning hook (GitGuardian/Trufflehog) must run to ensure no

API

keys, credentials, or certificates exist in the commit history.

- 5.5. The description must list:
 - The technical problem being solved.
 - Visual proof of functionality (UI screenshot or terminal logs).
 - Explicit instructions on how to test the code changes.

6. MANDATORY CHECKLIST FOR REVIEWERS

Reviewers must perform their validation against the following criteria:

6.1. Review Depth

- Reviewers must execute a full logical assessment of the code.
- Inspect variable naming, function complexity, and memory leaks.
- A simple syntax or spelling scan is strictly insufficient.

6.2. Review Duration

- A thorough code review must take at least 15 minutes of dedicated time.
- No PR containing more than 20 lines of logic changes may be approved in under 10 minutes. Doing so is classified as a process violation.

6.3. Automated Test Verification

- Reviewers must check the CI/CD dashboard to verify that all lint, security (SAST), and unit test pipelines have successfully passed.
- Bypassing or overriding failing automated checks is strictly forbidden.

6.4. Secrets and Credentials Check

- Reviewers must manually double-check that no API secrets, database connection strings, private keys, or passwords are included in the code.

6.5. Code Ownership and Dual Approval

- Submissions affecting Core Security Scope require dual approval.
- Approvals must be obtained from at least two senior engineering leads.

7. PROCEDURAL FLOW FOR CODE REVIEW

Step 1: Code Author creates a new feature branch and develops the change.

Step 2: Code Author executes all local tests and secret scans successfully.

Step 3: Code Author opens a Pull Request on Git, naming target reviewers.

Step 4: Automated CI pipeline runs security, linting, and build validation.

Step 5: Peer Reviewer claims the PR, sets a timer, and reviews line-by-line.

Step 6: Peer Reviewer leaves inline comments for improvements or queries.

Step 7: Code Author implements requested fixes and pushes commits.

Step 8: Peer Reviewer performs final checks, confirms 15-minute review minimum,

and marks the PR as "Approved".

Step 9: If Core Security is affected, secondary Senior Lead reviews and approves.

Step 10: Code Author merges PR. The branch is auto-deleted.

8. TROUBLESHOOTING & ESCALATION PROCEDURES

8.1. PR Approval Violations

- If an engineer approves a PR without running tests or reviewing security parameters, the system will raise an alert.
- The PR must be blocked immediately, and the issue must be escalated to the Engineering Manager for formal audit and corrective action.

8.2. Escalation Contact Path

- Incident Level: Code review process bypass.
- First Contact: Department Engineering Lead.
- Second Contact: Engineering Manager (Marcus Vance).
- VP of Engineering: Sneha Deshmukh.

9. REVISION HISTORY LOG

Version	Date	Author	Description of Change

v1.0	2024-03-10	M. Vance	Initial release of code review SOP.
v2.0	2024-11-22	M. Vance	Added secrets scanning guidelines.
v3.0	2025-06-15	S. Deshmukh	Mandated dual approvals on Core
Security.			
	v3.4	2026-01-15	M. Vance
